

Lecture 3/Module 2

CS3212 Algorithms

Aya Zirikly
09/01/2026

Review: Iterative Complexity

Topic Focus: Moving from iterative analysis to divide-and-conquer structures.

Iterative analysis refers to the technique used to determine the time complexity of non-recursive code—algorithms built on **sequential statements, conditionals, and loops** (like **for** and **while** loops)

Core Components of Iterative Analysis

Single Loops: Counting iterations directly proportional to input size.

- **Example:** A loop running n times gives $O(n)$ time.

Nested Loops: Multiplying loop bounds or setting up mathematical summations.

- **Example:** Two nested loops running up to n yield $\text{Sum}(i=1 \text{ to } n) \text{ Sum}(j=1 \text{ to } n) 1 = n^2 \Rightarrow O(n^2)$ time.

Core Components of Iterative Analysis

Dependent Loops: When an inner loop depends on the outer loop index (e.g., running from 1 to i as i goes from 1 to n), the total number of executions is the arithmetic series:

$$\text{Total Iterations} = 1 + 2 + 3 + \dots + n = n(n + 1) / 2 = (n^2 + n) / 2$$

```
def dependent_loops_example(n):  
    total_iterations = 0  
  
    # Outer loop runs from 1 to n  
    for i in range(1, n + 1):  
        # Inner loop depends on the outer loop index 'i'  
        for j in range(1, i + 1):  
            total_iterations += 1  
  
    return total_iterations
```

Outline

Topic Focus: Moving from iterative analysis to divide-and-conquer structures.

Agenda:

- The Divide-and-Conquer strategy template
- Mergesort: Mechanics, implementation, and trace
- Expressing runtime as recurrence relations
- Solving recurrences using the **Recursion Tree Method**

Divide-and-Conquer —Historical Roots

Human Calculators & Pre-Computer Math

- **Gauss (1805):** Invented the Fast Fourier Transform (FFT) to calculate asteroid orbits by hand. He split trigonometric series into even and odd indices, reducing work from $O(N^2)$ to $O(N \log N)$.

Early Hardware Constraints (1940s)

- **John von Neumann (1945):** Invented **Mergesort** for early computers like the EDVAC.
- **The Constraint:** Data was stored sequentially on slow magnetic tapes with limited memory.
- **The Solution:** Splitting data across streams and merging sorted chunks allowed computers to process massive datasets without keeping everything in memory.

Breaking Theoretical Bounds (1960s)

Overcoming the $O(n^2)$ Barrier

- **The Assumption:** Mathematician Andrey Kolmogorov conjectured that multiplying two n -digit numbers fundamentally required $O(n^2)$ operations (the grade-school method).
- **Karatsuba's Breakthrough (1960):** Anatolii Karatsuba split n -digit numbers into high and low halves. By recombining 3 sub-multiplications instead of 4, he achieved an $O(n^{1.58})$ algorithm, disproving Kolmogorov.

The Divide-and-Conquer Strategy Template

Break a problem into independent subproblems of the same type, solve them recursively, and combine their solutions.

The Steps:

1. **Divide:** Split problem of size n into smaller subproblems of size n/b .

2. **b is the subproblem reduction factor** (or subproblem size ratio).

3.

small.

It is the factor by which the input size n is divided at each step of the recursion, meaning each subproblem receives an input size of n/b .

For example:

- If **$b = 2$** , the input size is cut in half at each step (such as in Merge Sort or Binary Search).
- If **$b = 3$** , the input size is split into thirds at each step.

The Divide-and-Conquer Strategy Template

Break a problem into independent subproblems of the same type, solve them recursively, and combine their solutions.

The Steps:

1. **Divide:** Split problem of size n into smaller subproblems of size n/b .
2. **Conquer:** Solve subproblems recursively. Base case: directly solve when size is sufficiently small.

Base Case Examples:

- **Sorting (Mergesort/Quicksort):** An array of size 0 or 1 is already sorted → return immediately in $O(1)$ time.
- **Searching (Binary Search):** A range of size 1 (or $low > high$) → check single element or return "not found" in $O(1)$ time.

The Divide-and-Conquer Strategy Template

Break a problem into independent subproblems of the same type, solve them recursively, and combine their solutions.

1. **Divide:** Split problem of size n into smaller subproblems of size n/b .
2. **Conquer:** Solve subproblems recursively. Base case: directly solve when size is sufficiently small.
3. **Combine:** Merge subproblem solutions into the solution for the original problem.

Combine Examples:

- **Mergesort (Heavy Combine):** Interleave two sorted sub-arrays into one fully sorted array → requires active $O(n)$ work.
- **Quicksort (Trivial Combine):** Concatenate the sorted left side, pivot, and sorted right side → requires $O(1)$ extra work because elements are already in place.

The Divide-and-Conquer Strategy Template

$$T(n) = a * T(n/b) + D(n) + C(n)$$

Where:

- **$T(n)$** is the total running time for a problem of size n .
- **a** is the number of subproblems created during the divide step.
- **$T(n/b)$** is the time required to solve each subproblem of size n/b .
- **$D(n)$** is the time taken to divide the main problem into smaller subproblems.
- **$C(n)$** is the time taken to combine the subproblem solutions into the final answer.

$$T(n) = a * T(n/b) + f(n)$$

Real-World Applications

Spatial Data / GIS, Computer Graphics, Signal Processing

Map Navigation & Graphics Zooming (Quadtrees)

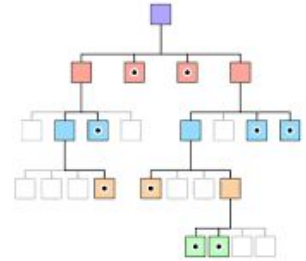
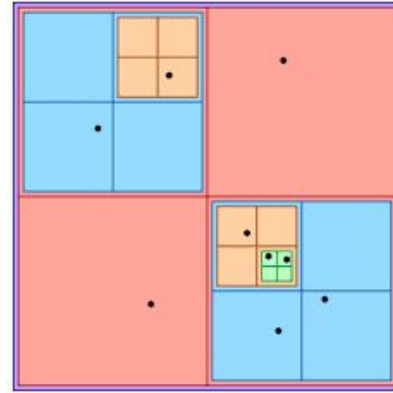
The Problem

Loading high-resolution map tiles for the entire globe simultaneously would overwhelm bandwidth and GPU memory.

Map Navigation & Graphics Zooming (Quadrees)

How the Strategy Template Applies

- **Divide:** Split a 2D viewport into 4 equal spatial quadrants (**NW**, **NE**, **SW**, **SE**). Unlike linear arrays where $b = 2$, spatial trees divide by $b = 4$.
- **Conquer:** Recursively process and subdivide **only** the visible quadrant as the user zooms in. Ignore off-screen quadrants entirely (*viewport culling*).
- **Combine:** Stitch the active sub-tiles together into a single seamless frame rendered at 60 FPS.



Map Navigation & Graphics Zooming (Quadrees)

Why It Matters for Algorithm Analysis

- **Non-Linear Branching:** Demonstrates that divide-and-conquer extends beyond 1D arrays to 2D spatial structures.
- **Logarithmic Scaling:** Search and render complexity drops from checking millions of map features ($O(N)$) to traversing tree height ($O(\log_4 N)$).

Merge Sort

Invention: Created by John von Neumann in 1945.

Core Idea: A fundamental, comparison-based sorting algorithm built on the **Divide-and-Conquer** design paradigm.

Primary Purpose: Efficiently orders collections by recursively breaking them down into smaller, easily solvable sub-problems.

Merge sort

1. Divide & Conquer

Divide: Partition the unsorted array of size n into two sub-arrays of size $n/2$ at the midpoint index.

Conquer: Recursively sort both sub-arrays until reaching trivial base cases of size 1 or 0.

2. Combine (Merge)

Combine: Merge the two sorted sub-arrays back together into a single unified sorted sequence.

Efficiency: Merging two pre-sorted lists requires linear time $O(n)$ using a two-pointer technique.

Steps

- ❑ **Divide:** Split the array of size n into two equal halves of size $n/2$.
- ❑ **Conquer:** Recursively sort both halves until ...

Steps

- ❑ **Divide:** Split the array of size n into two equal halves of size $n/2$.
- ❑ **Conquer:** Recursively sort both halves until reaching trivial base cases of size 1 or 0.

Steps

- ❑ **Divide:** Split the array of size n into two equal halves of size $n/2$.
- ❑ **Conquer:** Recursively sort both halves until reaching trivial base cases of size 1 or 0.
- ❑ **Combine:** Merge the two sorted halves (sub-arrays) into a single sorted array.
 - ❑ Efficiency: Merging two pre-sorted lists requires linear time $O(n)$ using a two-pointer technique.
- ❑ **Key Characteristic:** Work is heavily weighted in the **Combine** phase.
- ❑ **Base Case:** An array of size 0 or 1 is already sorted $O(1)$ time.

Example

Input Array: [38, 27, 43, 3, 9, 82, 10]

Divide Phase

- **Level 0 (Root Node):** [38, 27, 43, 3, 9, 82, 10]
- **Level 1 (Split into 2 sub-arrays):** Left Sub-array: [38, 27, 43] Right Sub-array: [3, 9, 82, 10]
- **Level 2 (Split into 4 sub-arrays):** From Left: [38] and [27, 43] From Right: [3, 9] and [82, 10]
- **Level 3 (Split into singletons / Base cases of size 1):**
 - a. From Left: [38], [27], [43]
 - b. From Right: [3], [9], [82], [10]

Example

Combine Phase

- **Step 1: Merge adjacent singletons into sorted pairs (Level 3 to Level 2)**
 - Merge [27] and [43] -> [27, 43]
 - Merge [3] and [9] -> [3, 9]
 - Merge [82] and [10] -> [10, 82]
 - Note: [38] remains a singleton for this turn.
 - **Resulting sub-arrays:** [38], [27, 43], [3, 9], [10, 82]
- **Step 2: Merge 2-element sub-arrays into 3-element and 4-element sorted halves (Level 2 to Level 1)**
 - Merge [38] and [27, 43] -> [27, 38, 43] (Left half fully sorted)
 - Merge [3, 9] and [10, 82] -> [3, 9, 10, 82] (Right half fully sorted)
 - **Resulting sub-arrays:** [27, 38, 43] and [3, 9, 10, 82]

Example

- **Step 3: Final Merge of the two sorted halves (Level 1 to Level 0)**
 - Merge [27, 38, 43] and [3, 9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
 - **Final Sorted Array:** [3, 9, 10, 27, 38, 43, 82]

Phase 1: Splitting Process

✂ **Constant Time Midpoint:** Splitting requires calculating $\text{mid} = (\text{left} + \text{right}) / 2$ taking $O(1)$ time per split.

👤 **Binary Tree Splitting:** The recursion continues until sub-arrays have 1 element (inherently sorted).

📦 **Logarithmic Height:** A list of size n splits into a tree depth of exactly $\log_2 n$ levels.




```
MERGE(A, p, q, r):  
    n1 = q - p + 1  
    n2 = r - q  
    create arrays L[1..n1+1] and R[1..n2+1]  
    copy A[p..q] to L, and A[q+1..r] to R  
    i = 1, j = 1  
    for k = p to r:  
        if L[i] <= R[j]:  
            A[k] = L[i]; i = i + 1  
        else:  
            A[k] = R[j]; j = j + 1
```

Formulating the Mergesort Recurrence

- $2T(n/2)$: Solving 2 subproblems of size $n/2$.
- $C \cdot n$: Linear time to divide array and merge sorted halves.

Question: How do we convert this recurrence into a closed-form bound?

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + c \cdot n & \text{if } n > 1 \end{cases}$$

Algebraic Solution

$$T(n) = 2 * T(n/2) + n$$

Step 1: Substitute $T(n/2)$ Since $T(n/2) = 2 * T(n/4) + (n/2)$:

$$T(n) = 2 * [2 * T(n/4) + (n/2)] + n$$

$$T(n) = 4 * T(n/4) + n + n$$

$$T(n) = 4 * T(n/4) + 2n$$

Step 2: Substitute $T(n/4)$ Since $T(n/4) = 2 * T(n/8) + (n/4)$:

$$T(n) = 4 * [2 * T(n/8) + (n/4)] + 2n$$

$$T(n) = 8 * T(n/8) + n + 2n$$

$$T(n) = 8 * T(n/8) + 3n$$

Algebraic Solution

$$T(n) = 2 * T(n/2) + n$$

Step 3: Find the general pattern after k steps

$$T(n) = (2^k) * T(n / (2^k)) + k * n$$

Algebraic Solution

$$T(n) = 2 * T(n/2) + n$$

Step 4: Reach the base case We stop dividing when the subproblem size becomes 1:

$$n / (2^k) = 1 \quad n = 2^k \quad k = \log_2(n)$$

Step 5: Substitute $k = \log_2(n)$ back into the pattern

$$T(n) = (2^{(\log_2(n))}) * T(1) + (\log_2(n)) * n$$

$$T(n) = n * T(1) + n * \log_2(n)$$

Final Result:

$$T(n) = O(n * \log_2(n))$$

The 4-Step Framework for Recursion Trees

Draw the Tree: Expand the recurrence relation down 3 to 4 levels to identify subproblem sizes and work performed at each individual node.

Calculate Work Per Level: Multiply the work done by a single node at level i by the total number of nodes present at level i .

Determine Tree Height (h): Solve for when the subproblem size shrinks to the base case size of 1: $n / (b^h) = 1$ which yields $h = \log_b(n)$.

Sum Across All Levels: Evaluate the geometric sum from level 0 through level h to compute the final asymptotic bound (**Big-O / Big-Theta**).

Solving Recurrences — The Recursion Tree Method

- ❑ Model recursive calls as a tree where nodes represent work done at that recursive level.
- ❑ Tree Mechanics for $T(n) = 2T(n/2) + cn$:
 - ❑ Root: Cost cn , branches into 2 children of size $n/2$.
 - ❑ Level 1: 2 nodes, each doing $c(n/2)$ work -> Level sum = cn .
 - ❑ Level 2: 4 nodes, each doing $c(n/4)$ work -> Level sum = cn .
 - ❑ Level i : 2^i nodes, each doing $c(n/2^i)$ work -> Level sum = cn .

Solving Recurrences — The Recursion Tree Method

- ❑ Tree Mechanics for $T(n) = 2T(n/2) + cn$:
 - ❑ Root: Cost cn , branches into 2 children of size $n/2$.
 - ❑ Level 1: 2 nodes, each doing $c(n/2)$ work -> Level sum = cn (work per node)
 - ❑ Level 2: 4 nodes, each doing $c(n/4)$ work -> Level sum = cn (work per node)
 - ❑ Level i : 2^i nodes, each doing $c(n/2^i)$ work -> Level sum = cn (work per node)
- ❑ Tree Height (h): Array size shrinks to 1 when $n / 2^h = 1 \Rightarrow h = \log_2(n)$.
- ❑ Total Work:

$$\text{Total} = \sum_{i=0}^{\log_2 n} cn = cn \cdot (\log_2 n + 1) = \Theta(n \log n)$$